

#1

Question: What index value does the third element of an array have?

Answer: 2

Java arrays start counting at 0, so:

- 1st element → index 0
 - 2nd element → index 1
 - 3rd element → index 2
-

#2

Question: Write a declaration for an array named `quantities` that stores 20 integers.

Answer:

```
int[] quantities = new int[20];
```

#3

Question: Write a declaration for an array named `heights` storing 1.65, 2.15, and 4.95.

Answer:

```
double[] heights = {1.65, 2.15, 4.95};
```

#4

Question: Write a for-each statement that displays the integer values stored in an array named `grades`.

Answer:

```
for (int grade : grades) {  
    System.out.println(grade);  
}
```

#5a

Question: Give an algorithm for inserting data into an array.

Answer:

1. Start at the last element currently being used.
 2. Move each element one position to the right.
 3. Continue until the insertion position is reached.
 4. Place the new value into the empty position.
 5. Increase the number of elements being used.
-

#5b

Question: Give an algorithm for deleting data from an array.

Answer:

1. Find the position of the element to delete.
 2. Start with the element immediately after it.
 3. Move each following element one position to the left.
 4. Continue until the last element has been moved.
 5. Decrease the number of elements being used.
-

#6

Question: How does passing an entire array to a method differ from passing a single element?

Answer:

When an entire array is passed to a method, the method can access the elements of the array.
When a single element is passed, only that individual value is passed to the method.

#8

Question: What is displayed?

```
String name = "Elaine";  
System.out.println(name.charAt(3));
```

Answer:

i

The indexes are:

E l a i n e
0 1 2 3 4 5

Therefore, `charAt(3)` is i.

#13 — True or False

a)

True

An array contains elements of the same data type.

b)

True

Java array indexes begin at 0.

c)

True

```
int[] empNums = new int[10];
```

creates 10 elements, indexed 0 through 9.

d)

False

An `int` array is automatically initialized to 0, not 1.

e)

True

An entire array can be passed to a method.

f)

True

`toCharArray()` converts a `String` into a `char` array.

g)

False

A linear search can search the entire array.

h)

False

```
int[][] grid = new int[4][4];
```

contains:

$4 \times 4 = 16$ elements

i)

False

Lowercase letters have higher Unicode values than their corresponding uppercase letters.

j)

False

The size of a normal Java array cannot be changed after it is created.

k)

True

`ArrayList` provides a dynamically sized collection.

l)

False

ArrayList stores objects, so primitive values such as `int` are represented using wrapper classes such as `Integer`.

#14 — Life

This is a programming exercise. The goal is to create Conway's Game of Life using a 20×20 grid.

```
public class Life {

    public static void main(String[] args) {

        boolean[][] world = new boolean[20][20];
        boolean[][] nextWorld = new boolean[20][20];

        // Example starting pattern
        world[9][10] = true;
        world[10][10] = true;
        world[11][10] = true;

        for (int generation = 0; generation < 10; generation++) {

            // Display the current generation
            for (int row = 0; row < 20; row++) {

                for (int col = 0; col < 20; col++) {

                    if (world[row][col]) {
                        System.out.print("X");
                    } else {
                        System.out.print(".");
                    }
                }

                System.out.println();
            }

            System.out.println();

            // Determine the next generation
            for (int row = 0; row < 20; row++) {
```

```

for (int col = 0; col < 20; col++) {

    int neighbors = 0;

    for (int r = row - 1; r <= row + 1; r++) {

        for (int c = col - 1; c <= col + 1; c++) {

            if (r >= 0 && r < 20 &&
                c >= 0 && c < 20 &&
                !(r == row && c == col) &&
                world[r][c]) {

                neighbors++;
            }
        }
    }

    if (world[row][col]) {

        if (neighbors == 2 || neighbors == 3) {
            nextWorld[row][col] = true;
        } else {
            nextWorld[row][col] = false;
        }

    } else {

        if (neighbors == 3) {
            nextWorld[row][col] = true;
        } else {
            nextWorld[row][col] = false;
        }
    }
}

// Copy the next generation
for (int row = 0; row < 20; row++) {

    for (int col = 0; col < 20; col++) {
        world[row][col] = nextWorld[row][col];
    }
}

```

```
}  
}  
}  
}
```

#15 — Mankala

This exercise requires a two-player Mankala game using an array to represent the board.

```
import java.util.Scanner;  
  
public class Mankala {  
  
    public static void main(String[] args) {  
  
        Scanner input = new Scanner(System.in);  
  
        int[] board = {  
            3, 3, 3, 3, 3, 3, 0,  
            3, 3, 3, 3, 3, 3, 0  
        };  
  
        int player = 1;  
        boolean gameOver = false;  
  
        while (!gameOver) {  
  
            displayBoard(board);  
  
            System.out.println("Player " + player + "'s turn.");  
            System.out.print("Choose a pit (A-F): ");  
  
            String choice = input.next().toUpperCase();  
  
            int pit = choice.charAt(0) - 'A';  
  
            if (player == 2) {  
                pit += 7;  
            }  
  
            if (board[pit] == 0) {  
                System.out.println("That pit is empty.");  
            }  
        }  
    }  
}
```

```

        continue;
    }

    int stones = board[pit];
    board[pit] = 0;

    int position = pit;

    while (stones > 0) {

        position++;

        if (position == 14) {
            position = 0;
        }

        // Skip opponent's home
        if (player == 1 && position == 13) {
            continue;
        }

        if (player == 2 && position == 6) {
            continue;
        }

        board[position]++;
        stones--;
    }

    // Check for another turn
    if (!((player == 1 && position == 6) ||
        (player == 2 && position == 13))) {

        if (player == 1) {
            player = 2;
        } else {
            player = 1;
        }
    }

    boolean player1 = false;
    boolean player2 = false;

    for (int i = 0; i < 6; i++) {

```

```

        if (board[i] > 0) {
            player1 = true;
        }
    }

    for (int i = 7; i < 13; i++) {
        if (board[i] > 0) {
            player2 = true;
        }
    }

    if (!player1 || !player2) {
        gameOver = true;
    }
}

// Move remaining stones to home
for (int i = 0; i < 6; i++) {
    board[6] += board[i];
    board[i] = 0;
}

for (int i = 7; i < 13; i++) {
    board[13] += board[i];
    board[i] = 0;
}

displayBoard(board);

System.out.println("Player 1 score: " + board[6]);
System.out.println("Player 2 score: " + board[13]);

input.close();
}

public static void displayBoard(int[] board) {

    System.out.println();

    System.out.println("Player 2: "
        + board[12] + " "
        + board[11] + " "
        + board[10] + " "
        + board[9] + " "

```

```
        + board[8] + " "
        + board[7]);

System.out.println("Home: " + board[13]
        + "
        "
        + board[6]);

System.out.println("Player 1: "
        + board[0] + " "
        + board[1] + " "
        + board[2] + " "
        + board[3] + " "
        + board[4] + " "
        + board[5]);

System.out.println("      A B C D E F");
System.out.println();
    }
}
```